Prometheus ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Prometheus?
Prometheus is an **open-source monitoring and alerting tool** used to collect and store metrics from applications and infrastructure.
It collects metrics such as CPU usage, memory usage, request count, response time, and error rate.
Prometheus stores these metrics as **time-series data**, meaning values are stored along with timestamps.
Simple ga gurthupettuko:
Prometheus = **Metrics collect + Store + Query + Alert**
Eng: Prometheus is an open-source monitoring system that collects, stores, queries, and alerts on time-series metrics.

---

2. Why do we use Prometheus?
In a production environment, we need to know whether applications and infrastructure are healthy.
Prometheus helps us monitor things like:

- CPU and memory usage
- Application requests
- Error rates
- Response time
- Pod/container health
- Infrastructure metrics

If a metric crosses a defined threshold, Prometheus can trigger an alert.
Simple ga gurthupettuko:
System → Metrics → Prometheus → Monitoring → Alert
Eng: Prometheus is used to monitor infrastructure and applications by collecting metrics and generating alerts when defined conditions occur.

---

3. What is a Metric in Prometheus?
A metric is a numerical measurement that represents the state or activity of a system.
Examples:
CPU Usage
Memory Usage
HTTP Requests
HTTP Errors
Request Duration
Active Connections
For example:
http_requests_total 15000
means the application has recorded a particular HTTP request count.
Simple ga gurthupettuko:
Metric = **System/Application measurement.**
Eng: A Prometheus metric is a numerical time-series measurement representing some aspect of system or application behavior.

---

4. What is Time-Series Data?
Time-series data is data collected over time.
For example:
10:00 → CPU = 40%
10:01 → CPU = 55%
10:02 → CPU = 70%
10:03 → CPU = 45%
Prometheus stores metric values along with timestamps and labels.
Simple ga gurthupettuko:
Time-Series = **Value + Time**
Eng: Time-series data represents metric values recorded at different points in time.

---

5. What is a Prometheus Target?
A target is an endpoint from which Prometheus collects metrics.
For example:
Application
↓
/metrics
↓
Prometheus Target
A target may be an application, server, Kubernetes workload, or exporter exposing Prometheus-compatible metrics.
Simple ga gurthupettuko:
Target = **Prometheus metrics collect chese endpoint.**
Eng: A Prometheus target is an endpoint that Prometheus scrapes to collect metrics.

6. What is Scraping in Prometheus?
Scraping means Prometheus periodically **pulls metrics from a target's HTTP endpoint**.
For example:
Prometheus
↓ HTTP GET
Application /metrics
↓
Metrics
Prometheus repeatedly collects the metrics based on its configured scrape interval.
Simple ga gurthupettuko:
Scraping = **Prometheus target ni periodic ga metrics kosam query cheyyadam.**
Eng: Scraping is the process by which Prometheus periodically retrieves metrics from configured targets.

---

7. Why does Prometheus use Pull Model?
Prometheus normally uses a **pull-based monitoring model**.
Instead of applications continuously sending metrics to Prometheus, Prometheus periodically contacts the target and retrieves the metrics.
This makes monitoring centralized and allows Prometheus to determine whether a target is reachable.
Simple ga gurthupettuko:
Pull = Prometheus → Target → Metrics
Eng: Prometheus primarily uses a pull model where the Prometheus server periodically retrieves metrics from monitored targets.

---

8. What is the difference between Pull and Push monitoring?
In **Pull model**, Prometheus contacts the target and retrieves metrics.
Prometheus → Target
In **Push model**, the application or agent sends metrics to a monitoring system.
Application → Monitoring System
Prometheus primarily uses Pull, although components such as Pushgateway can be used for specific short-lived job scenarios.
Simple ga gurthupettuko:
Pull = Monitoring system collects
Push = Application sends
Eng: Prometheus primarily follows a pull-based model, while push-based mechanisms can be used for specific use cases.

---

9. What is /metrics **endpoint?**
/metrics is a commonly used HTTP endpoint where an application or exporter exposes metrics in a Prometheus-readable format.
Example:
http://application:8080/metrics
Prometheus scrapes this endpoint and stores the returned metrics.
Simple ga gurthupettuko:
/metrics = **Metrics available unna endpoint.**
Eng: The /metrics endpoint commonly exposes application or infrastructure metrics in a format that Prometheus can scrape.

---

10. What is an Exporter in Prometheus?
An Exporter is a component that collects metrics from a system and exposes them in a format Prometheus can scrape.
For example, **Node Exporter** exposes Linux system metrics such as CPU, memory, disk, and network statistics.
Linux Server
↓
Node Exporter
↓
/metrics
↓
Prometheus
Simple ga gurthupettuko:
Exporter = **System metrics ni Prometheus-readable format lo expose chestundi.**
Eng: An Exporter exposes metrics from systems or applications in a format that Prometheus can scrape.

---

11. What is Node Exporter?
Node Exporter is used to expose hardware and operating-system-level metrics from Unix/Linux systems.
It can provide metrics related to:

- CPU
- Memory
- Disk
- Filesystem
- Network
- Load

Simple ga gurthupettuko:
Node Exporter = **Linux server metrics.**
Eng: Node Exporter exposes operating-system and hardware-related metrics from Unix-like systems for Prometheus monitoring.

12. How does Prometheus monitor a Linux server?
A common architecture is:
Linux Server
↓
Node Exporter
↓
/metrics
↓
Prometheus
↓
Grafana
Node Exporter collects system metrics and exposes them. Prometheus scrapes those metrics and stores them.
Grafana can then visualize them.
Simple ga gurthupettuko:
Server → Node Exporter → Prometheus → Grafana
Eng: A Linux server can be monitored using Node Exporter, which exposes system metrics that Prometheus scrapes and stores for querying and visualization.

---

13. What is PromQL?
PromQL stands for **Prometheus Query Language**.
It is used to query and analyze metrics stored in Prometheus.
Example:
up
This can be used to check the up/down status of targets.
Another example:
rate(http_requests_total[5m])
This calculates the per-second rate of increase over the selected five-minute range.
Simple ga gurthupettuko:
PromQL = **Prometheus metrics query cheyyadaniki language.**
Eng: PromQL is Prometheus's query language used to select, calculate, aggregate, and analyze time-series metrics.

---

14. What does the up **metric mean?**
The up metric is commonly used to indicate whether Prometheus successfully scraped a target.
Typically:
up = 1 → Target scrape successful
up = 0 → Target scrape failed
It is very useful for basic target availability monitoring.
Simple ga gurthupettuko:
up=1 → Reachable
up=0 → Scrape failed
Eng: The up metric indicates whether Prometheus successfully scraped a target during the latest scrape.

---

15. What are Labels in Prometheus?
Labels are key-value pairs attached to metrics that provide additional dimensions.
Example:
```
http_requests_total{
  method="GET",
  status="200",
  service="payment"
}
```
Labels allow us to filter and aggregate metrics by service, status, method, environment, and other dimensions.
Simple ga gurthupettuko:
Labels = **Metric ki additional information/dimensions.**
Eng: Prometheus labels provide dimensions that allow metrics to be filtered and aggregated by attributes such as service, environment, method, or status.

---

16. Why are Labels important in Prometheus?
Suppose we monitor 20 services.
Without labels, it becomes difficult to identify which service generated a metric.
With labels:
service="payment"
service="orders"
service="users"
we can query individual services or aggregate metrics across them.
Simple ga gurthupettuko:
Labels → **Filter + Group + Compare metrics**
Eng: Labels make metrics multidimensional and allow effective filtering, grouping, and aggregation in PromQL.

17. What is Cardinality in Prometheus?
Cardinality refers to the number of unique combinations of label values for a metric.
High-cardinality labels can create a very large number of time series and increase memory and storage requirements.
For example, using a unique user_id or request ID as a label can create extremely high cardinality.
Simple ga gurthupettuko:
Too many unique labels → **Too many time series → Performance problem**
Eng: Metric cardinality is the number of unique label combinations, and uncontrolled high cardinality can significantly increase Prometheus resource usage.

---

18. Why should we avoid high-cardinality labels?
Suppose an application has millions of users.
If we create:
http_requests_total{user_id="123456"}
for every user, Prometheus may have to maintain a huge number of unique time series.
This can consume significant memory and storage.
Instead, we should use useful bounded labels such as service, HTTP method, or status code where appropriate.
Simple ga gurthupettuko:
User ID / Request ID labels ❌
Low-cardinality useful labels ✅
Eng: High-cardinality labels should be avoided because they can create excessive time series and negatively affect Prometheus performance and resource consumption.

---

19. What is Service Discovery?
Service Discovery allows Prometheus to automatically discover monitoring targets instead of manually configuring every target.
In Kubernetes environments, Prometheus can discover Pods, Services, Nodes, and other resources using Kubernetes APIs and discovery mechanisms.
Kubernetes
↓
Service Discovery
↓
Prometheus
↓
Targets
Simple ga gurthupettuko:
Service Discovery = **Targets ni automatic ga find cheyyadam.**
Eng: Service Discovery allows Prometheus to dynamically discover monitoring targets as infrastructure changes.

---

20. Why is Service Discovery important in Kubernetes?
Kubernetes environments are dynamic.
Pods are created, deleted, and replaced frequently, and their IP addresses can change.
Manually maintaining every Pod IP in Prometheus configuration is not practical.
Service Discovery automatically identifies relevant targets.
Simple ga gurthupettuko:
Pods dynamic → Manual IP management ❌ → Service Discovery ✅
Eng: Service Discovery is important in Kubernetes because workloads are dynamic and monitoring targets can change frequently.

---

21. What is Prometheus Alerting?
Prometheus can evaluate alerting rules against metrics.
For example:
CPU > 80%
for a defined period can trigger an alert.
Prometheus can send alert information to **Alertmanager**, which handles notification routing.
Simple ga gurthupettuko:
Metric → Alert Rule → Alertmanager → Notification
Eng: Prometheus alerting evaluates defined rules against metrics and can send firing alerts to Alertmanager for notification handling.

---

22. What is Alertmanager?
Alertmanager is a component used with Prometheus to **handle and route alerts**.
It can group related alerts, suppress duplicate notifications, and route notifications to configured receivers.
For example:
Prometheus
↓
Alertmanager
↓
Email / Slack / Pager / Other Receiver
Simple ga gurthupettuko:
Prometheus = Alert detect
Alertmanager = **Alert manage + route**
Eng: Alertmanager handles alerts received from Prometheus by grouping, deduplicating, suppressing, and routing notifications.

23. Difference between Prometheus and Alertmanager?
Prometheus is responsible for:
Metrics
Scraping
Storage
Querying
Alert Rule Evaluation
Alertmanager is responsible for:
Alert Routing
Grouping
Deduplication
Silencing
Notifications
Simple ga gurthupettuko:
Prometheus = **Detect**
Alertmanager = **Manage + Notify**
Eng: Prometheus detects alert conditions, while Alertmanager manages and routes the resulting alerts to notification systems.

---

24. How do you monitor CPU usage using Prometheus?
First, I install/configure Node Exporter on the server.
Node Exporter exposes CPU metrics.
Prometheus scrapes those metrics and stores them.
Then I use PromQL to calculate CPU utilization and visualize it in Grafana or use it in an alert rule.
Simple ga gurthupettuko:
Server → Node Exporter → Prometheus → PromQL → Grafana/Alert
Eng: I monitor CPU by collecting system metrics through Node Exporter, scraping them with Prometheus, and querying or alerting on the resulting time-series data.

---

25. How do you troubleshoot a target showing DOWN in Prometheus?
First I check the Prometheus **Targets** page.
If the target is down, I verify:
Target IP / DNS
↓
Port
↓
/metrics endpoint
↓
Network connectivity
↓
Exporter/Application status
↓
Prometheus configuration
I also check the scrape error shown by Prometheus because it often indicates whether the issue is connection refused, timeout, DNS, authentication, or an invalid metrics endpoint.
Simple ga gurthupettuko:
Target DOWN → Endpoint → Port → Network → Exporter → Config
Eng: I troubleshoot a down target by checking the target status, scrape error, endpoint, port, network connectivity, exporter/application health, and Prometheus configuration.

---

26. What happens if Node Exporter goes down?
Prometheus will fail to scrape the Node Exporter endpoint.
The target will typically show:
up = 0
and related alerts can fire if configured.
I would check whether Node Exporter is running, whether its port is listening, and whether network connectivity from Prometheus is working.
Simple ga gurthupettuko:
Node Exporter DOWN → Scrape Fail → up=0 → Alert
Eng: If Node Exporter goes down, Prometheus cannot collect the server metrics, so the target becomes unavailable and corresponding alerts may fire.

---

27. What happens if Prometheus itself goes down?
Prometheus will stop scraping targets and new metrics will no longer be collected by that Prometheus instance.
Existing data remains available only according to what has already been stored and retained.
In Production, we should consider appropriate availability, durability, retention, and recovery architecture based on monitoring requirements.
Simple ga gurthupettuko:
Prometheus DOWN → New metrics collection stops.
Eng: If Prometheus goes down, its scraping and new metric collection stop, so Production monitoring should include appropriate availability and recovery planning.

---

28. Prometheus vs CloudWatch — what is the difference?
Prometheus is commonly used for **metrics monitoring with PromQL**, especially in Kubernetes and cloud-native environments.
AWS CloudWatch is AWS's native monitoring service and provides metrics, logs, alarms, dashboards, and AWS-service integration.
A company may use both.
AWS Services → CloudWatch
Kubernetes/App Metrics → Prometheus
Simple ga gurthupettuko:
CloudWatch = AWS Native Monitoring
Prometheus = **Cloud-Native Metrics Monitoring**
Eng: Prometheus is a flexible time-series monitoring system commonly used for application and Kubernetes metrics, while CloudWatch provides native monitoring and observability capabilities across AWS services.

---

29. Prometheus vs Grafana — what is the difference?
Prometheus is primarily responsible for **collecting, storing, and querying metrics**.
Grafana is primarily used for **visualizing and exploring metrics** from Prometheus and other data sources.
Prometheus → Metrics
↓
Grafana → Dashboards
Simple ga gurthupettuko:
Prometheus = **Data**
Grafana = **Visualization**
Eng: Prometheus collects and stores metrics, while Grafana provides dashboards and visualization for those metrics.

---

30. Explain a real-time Prometheus monitoring architecture.
Suppose we have a Kubernetes application.
Application Pods expose application metrics. Prometheus discovers the required targets and scrapes the metrics.
Prometheus stores the time-series data. Engineers use PromQL to query the data and Grafana to visualize it.
If an important metric crosses a threshold, Prometheus evaluates the alert rule and sends the alert to Alertmanager, which routes the notification.
Kubernetes / Applications
↓
Service Discovery
↓
Prometheus
↓
Time-Series
↓
PromQL
↓
Grafana

Prometheus
↓
Alert Rules
↓
Alertmanager
↓
Notification
Simple ga gurthupettuko:
Targets → Scrape → Prometheus → PromQL → Grafana
Alert → Alertmanager → Notification
Eng: In a real-time setup, Prometheus dynamically discovers application targets, scrapes and stores their metrics, PromQL is used for analysis, Grafana provides visualization, and Alertmanager handles notifications for firing alerts.